# CPE 663: Special Topic III:

# Multilingual NLP and Low-Resource NLP

## Major Assignment 3

## Mini Transformer Benchmark Report

**A Small Transformer Encoder for Synthetic Sequence Classification**


## Member

67070701603 Kantapat Suwannahong


This report is submitted as part of Major Assignment 3: Mini Transformer Benchmark for the course CPE 663: Special Topic III: Multilingual NLP and Low Resource NLP.

Semester 2, Academic Year 2025

[[GITHUB REPOSITORY]/MA03_CPE663-2-2025_Mini-Transformer_Benchmark

## 1. Introduction

The objective of this assignment is to understand the core mechanisms of the Transformer architecture by implementing a small Transformer encoder from scratch and evaluating it on a synthetic sequence classification task. The goal is to study how token embeddings, positional information, attention, masking, and encoder blocks work together in a Transformer-based classifier.

## 2. Method

In this assignment, I implemented a mini Transformer encoder model for a binary sequence classification task. The general direction of the work was defined by the assignment: the model had to use the provided dataset, implement the main Transformer components from scratch, train and evaluate the model, and compare several model variants.

### 2.1. Task Definition

Each input is a sequence of discrete tokens with a valid length between 6 and 20 tokens. For batch processing, shorter sequences are padded so that all model inputs have a fixed length of 20 tokens, as required by the assignment. The goal is to predict whether the first non-padding token appears again in the second half of the valid sequence. If the first token appears in the second half, the label is 1. If it does not appear, the label is 0.

### 2.2. Input Representation

The vocabulary was specified by the assignment: PAD = 0, A = 1, B = 2, C = 3, and D = 4. Sequences vary from 6 to 20 tokens, standardized to 20 by padding with PAD tokens for batch training. An attention mask indicates real tokens with 1 and padding tokens with 0. A token embedding layer converts token IDs into 64-dimensional dense vectors, balancing effective learning with training speed. To avoid interference from padding, the attention mask is utilized to ensure the Transformer does not consider padding positions during processing.

### 2.3. Transformer Encoder Architecture

In this section, I created a mini Transformer using basic PyTorch layers. I used learned positional embeddings. The assignment required positional encoding but did not specify one exact type, and I chose this approach because it is simple to implement and suitable for a fixed maximum sequence length of 20. Positional information is important in this task because the model must determine whether the first token appears again in the second half of the valid sequence. Each encoder block contains multi-head self-attention followed by a feed-forward network. In the attention module, query, key, and value vectors are produced by linear layers, and the attention weights are computed with scaled dot-product attention. The feed-forward network uses two linear layers with GELU activation. I used a feed-forward dimension and dropout rate by following the assignment suggestions, since these values are appropriate for a small Transformer benchmark. Residual connections and layer normalization were applied around both sub-layers to support stable training.

### 2.4. Padding Mask Handling

Padding mask handling was required by the assignment. Since all sequences are padded to length 20, PAD tokens should not affect the attention calculation. To handle this, I used the provided attention mask and replaced the attention scores for padding positions with a very negative value before applying SoftMax. As a result, the model attends only to valid tokens and ignores padded positions.

### 2.5. Pooling and Classification Head

The assignment allowed several pooling choices and recommended mean pooling for simplicity. However, I used first-token pooling because the label rule is directly related to the first non-padding token. Through self-attention, the first token representation can gather information from the rest of the valid sequence, making it suitable for this task.

After pooling, the sequence representation is passed into a classification head with layer normalization, a linear layer, GELU activation, dropout, and a final linear layer. The output has two logits, one for label 0 and one for label 1.

## 3. Experimental Setup

In this assignment, I followed the recommended settings in most cases because they are suitable for a small-scale experiment and help keep the benchmark fair across model variants. The benchmark was designed to examine the effect of positional encoding, the number of attention heads, and the number of encoder layers.

### 3.1. Dataset and Data Split

The assignment required the use of the provided dataset files. Therefore, I used train.csv, validation.csv, and test.csv.

### 3.2. Model Variants

The assignment required a benchmark with at least three model settings, including: a model with positional encoding, a model without positional encoding, a comparison with different numbers of attention heads. To satisfy these requirements, I evaluated four model variants: **Model A**: positional encoding, 1 attention head, 1 encoder layer, **Model B**: positional encoding, 4 attention heads, 1 encoder layer, **Model C**: no positional encoding, 4 attention heads, 1 encoder layer, **Model D**: positional encoding, 4 attention heads, 2 encoder layers

I included four variants instead of only three because this gives a clearer comparison. Models A and B show the effect of changing the number of heads. Models B and C show the effect of removing positional encoding. Models B and D show the effect of adding one more encoder layer.

### 3.3. Hyperparameter Settings

The main hyperparameters used in the experiments were: embedding dimension = 64, feed-forward dimension = 128, number of attention heads = 1 or 4 depending on the model variant, number of encoder layers = 1 or 2 depending on the model variant, dropout = 0.1, batch size = 32, learning rate = 0.001, weight decay = 0.0001, and number of epochs = 10. I also used first-token pooling in the classifier.

### 3.4. Training Procedure

The model was trained using cross-entropy loss and optimized with AdamW. During training, I recorded the training loss and validation performance at each epoch. The best model state was selected based on validation F1 score and then used for evaluation on the test set.

### 3.5. Reproducibility

The assignment required reproducibility, so I fixed the random seed in the experiments. This helps reduce variation between runs and makes the comparison between model variants more meaningful.

### 3.6. Evaluation Metrics

The assignment required reporting validation accuracy, test accuracy, training time, and parameter count for each model. Therefore, I reported these values in the benchmark table. In addition, I also reported the F1 score because the dataset is imbalanced toward label 1, so accuracy alone may not fully reflect model quality. I also plotted validation accuracy by epoch to visualize the learning process. This was my own addition to the analysis.

## 4. Results

This section presents the benchmark results of the implemented mini Transformer models. The purpose of the benchmark was to compare the effect of positional encoding, the number of attention heads, and the number of encoder layers.

### 4.1. Benchmark Summary

The benchmark results are shown in Table 1. Among the four models, Model B achieved the best overall performance. It produced the highest validation accuracy, the highest test accuracy, and the highest test F1 score. Model D also performed strongly, but its results were slightly lower than Model B while using more parameters and longer training time.

Table 1. Benchmark results of the mini Transformer variants.

| Model | PE | Heads | Layers | Params | Time (s) | Val Acc | Test Acc | Test F1 |
|---|---|---|---|---|---|---|---|---|
| A_pos_1head_1layer | Yes | 1 | 1 | 39,490 | 75.9 | 0.9350 | 0.9160 | 0.9509 |
| B_pos_4head_1layer | Yes | 4 | 1 | 39,490 | 90.1 | 0.9880 | 0.9800 | 0.9879 |
| C_no_pos_4head_1layer | No | 4 | 1 | 38,210 | 72.4 | 0.8550 | 0.8670 | 0.9236 |
| D_pos_4head_2layer | Yes | 4 | 2 | 72,962 | 99.2 | 0.9830 | 0.9700 | 0.9819 |

### 4.2. Effect of Positional Encoding

To examine the effect of positional encoding, I compared Model B and Model C. These two models have the same number of heads and layers, but Model B uses positional encoding while Model C does not.

The results show a clear performance drop when positional encoding is removed. Model B achieved a test accuracy of 0.980, while Model C achieved 0.867. The test F1 score also decreased from 0.9879 to 0.9236. This suggests that positional information is very important because the model must determine whether the first token appears in the second half of the valid sequence.

### 4.3. Effect of the Number of Attention Heads

To examine the effect of changing the number of attention heads, I compared Model A and Model B. Both models use positional encoding and one encoder layer, but Model A uses 1 head while Model B uses 4 heads.

The results show that increasing the number of heads improved performance. Model A achieved a test accuracy of 0.916, while Model B achieved 0.980. The test F1 score also increased from 0.9509 to 0.9879. This suggests that multi-head attention helped the model capture useful token relationships more effectively.

### 4.4. Effect of the Number of Encoder Layers

I also compared a one-layer and a two-layer model by examining Model B and Model D. Both models use positional encoding and 4 heads, but Model B has 1 encoder layer and Model D has 2 encoder layers.

The two-layer model still performed well, with a test accuracy of 0.970 and a test F1 score of 0.9819. However, it did not outperform Model B, which achieved slightly better results with fewer parameters and shorter training time. This suggests that one encoder layer was already sufficient for this task.

### 4.5. Training Curve

Figure 1 shows the validation accuracy of all four model variants across training epochs. The models with positional encoding generally learned faster and reached higher validation accuracy than the model without positional encoding. In particular, Model B and Model D achieved strong validation performance early in training and remained stable in later epochs.

In contrast, Model C improved more slowly and stayed at a lower validation accuracy level throughout training. Model A also improved steadily, but its final validation accuracy remained lower than Model B. Overall, Figure 1 supports the main findings in the benchmark results table (Table 1).

To complement the validation accuracy curve, I also examined the training loss using a logarithmic y-axis, as shown in Figure 2. This view makes the early rapid decrease in loss and the later smaller improvements easier to observe. The log-scale loss curve shows that the stronger models reduced training loss quickly and then continued to improve more gradually during later epochs.

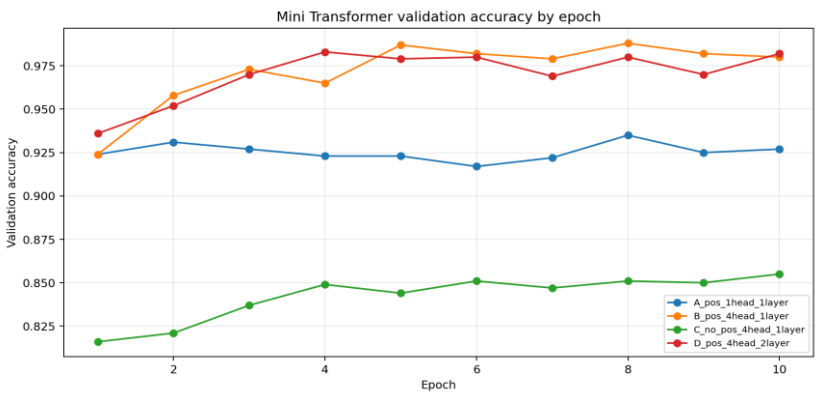


Figure 1. Validation accuracy curves across training epochs.

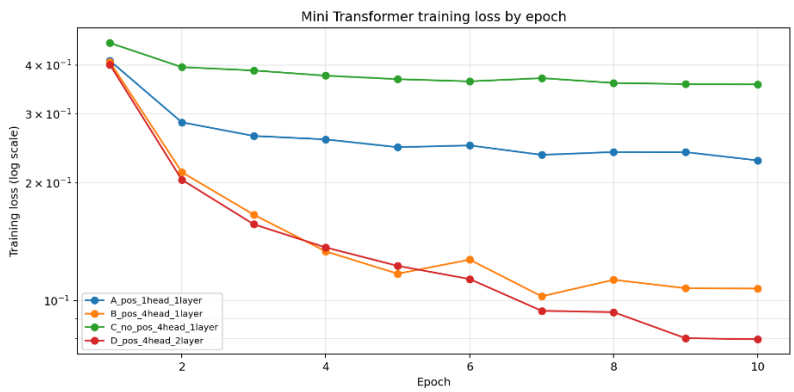


Figure 2. Training loss curves with a logarithmic y-axis across training epochs.

## 5. Discussion

The benchmark results show that the implemented mini Transformer was able to solve the synthetic sequence classification task effectively, but the model design choices had a clear effect on performance. Positional encoding and the number of attention heads were important factors, while increasing the number of encoder layers did not always lead to better results.

One important finding is that positional encoding had a strong effect on model quality. When I compared the model with positional encoding and the model without positional encoding, I observed a clear decline in both validation and test performance after positional encoding was removed. This result is reasonable because the task depends on both token identity and token order.

Another important finding is that using more attention heads improved performance. The model with 4 attention heads performed better than the model with only 1 head, even when the other main settings were the same. A likely explanation is that multiple heads allow the model to learn several token relationships at the same time.

The comparison between the one-layer and two-layer models showed that a deeper model was not necessary for this task. Since the dataset is small and the target rule is relatively simple, one encoder layer may already provide enough representational power. The extra layer increased parameter counts and training time but did not produce a clear improvement in final performance.

The results also support the importance of padding mask handling and the use of first-token pooling. Padding masking ensures that the model focuses only on valid tokens, while first-token pooling works well because the label depends directly on the first non-padding token.

Overall, the experimental results are consistent with the expected behavior of Transformer models. Positional information is important when the task depends on token order, multi-head attention improves the model's ability to capture useful relationships, and larger models do not always perform better on smaller and simpler tasks.

## 6. Conclusion

In this assignment, I implemented a mini Transformer encoder from scratch for a synthetic sequence classification task. The main goal was not only to obtain good performance, but also to understand the internal structure of the Transformer architecture through implementation and experimentation.

The experimental results showed that the model was able to solve the task effectively. Positional encoding and the use of multiple attention heads improved performance, while increasing the number of encoder layers did not provide a clear advantage for this small task.

Overall, this assignment helped me better understand both the theory and the practical behavior of Transformer encoders. It showed that even in a small and controlled problem setting, the main Transformer components play an important role in learning sequence patterns.

## 7. Acknowledgment

I used AI-based assistance for translation, language refinement, and identifying unclear or weak parts of the report. I also consulted the PyTorch documentation, Hugging Face resources, freecodecamp and Machine Learning Mastery to support my understanding during this assignment.